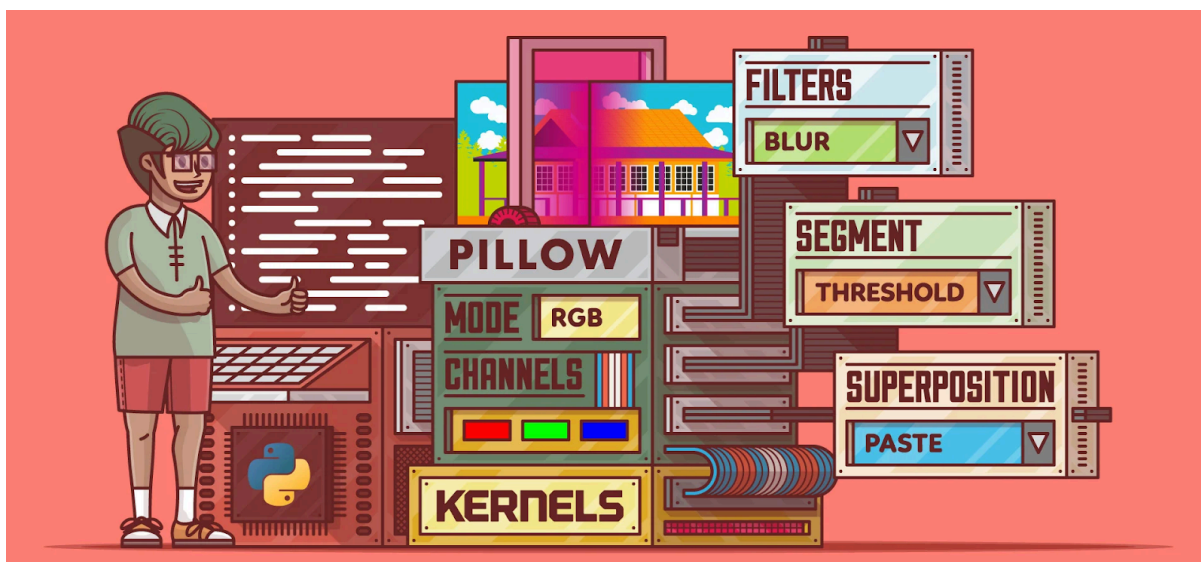


AirMouse Project (paper tecnico)



Indice

AirMouse Project (paper tecnico)	1
Architettura del Sistema	3
Analisi del moduli	3
webcam_selector.....	3
Funzioni Chiave.....	3
Dettagli implementativi.....	4
shared_state.....	4
Struttura.....	4
Utilizzo.....	4
settings_gui.....	5
Struttura.....	5
Elementi Grafici.....	5
Dettagli Implementativi.....	5
run_tracker.....	6
Flusso Principale.....	6
Elaborazione Frame.....	6
Gestione Gesture.....	6

Ottimizzazioni.....	6
Terminazione.....	6
gui_theme.....	7
Parametri.....	7
config.....	7

Architettura del Sistema

Il sistema è composto da 6 file:

webcam_manager.py: Selezione della webcam tramite GUI.

shared_state.py: Gestione dello stato globale (esecuzione/non esecuzione).

settings_gui.py: Configurazione parametri di controllo.

hand_tracker.py: Core dell'applicazione (elaborazione frame e gesture).

mouse_controller.py: Controllo del puntatore e gestione eventi

gesture_recognizer.py: Logica di riconoscimento gesti

gui_theme.py: Standardizzazione interfaccia grafica

config.py: Parametri di configurazione

Analisi del moduli

webcam_selector

webcam_manager.py

Identifica tutte le webcam disponibili (massimo 2) e permette all'utente di selezionarne una tramite GUI. Vanno bene anche webcam wireless.

Verifica a 3 livelli con apertura dispositivo, acquisizione frame, naming, inoltre è presente una gestione errori completa con fallback a ID numerico.

Funzioni Chiave

get_camera_names(): Esegue v4l2-ctl (Linux) per ottenere i nomi delle periferiche video.

try_camera(i, found, camera_names): Prova ad aprire la webcam con indice i usando OpenCV (cv2.VideoCapture). Se riuscito, aggiunge la webcam alla lista found.

select_camera_gui(): Crea una finestra Tkinter con pulsanti dinamici per ogni webcam rilevata. Utilizza thread separati per la ricerca per evitare il blocco della GUI.

Dettagli implementativi

Multi-threading: La ricerca delle webcam avviene in un thread separato (threading.Thread).

Gestione Errori: Verifica tramite cv2.VideoCapture.isOpened() e mostra messaggi di errore con messagebox.

Sincronizzazione: Utilizza threading.Event (stop_search) per interrompere la ricerca se l'utente seleziona una webcam.

shared_state

shared_state.py

Fornisce uno stato globale condiviso tra i moduli tramite threading.Event. , e viene utilizzato da (settings_gui.py , app.py, webcam_manager.py)

Struttura

running_event: Oggetto Event che segnala se l'applicazione è in esecuzione.

set_running(value): Imposta lo stato (True/False).

get_running(): Restituisce lo stato corrente.

Utilizzo

Controlla se l'applicazione deve terminare (es. quando l'utente chiude una finestra).

Sincronizza il thread della GUI con il loop principale in run_tracker.py.

settings_gui

settings_gui.py

Interfaccia grafica per modificare parametri di controllo e visualizzare istruzioni sulle gesture con 15 parametri configurabili e gestione dei profili.

Struttura

save_settings(): Scrive i valori correnti in Profiles/settings.txt.

load_settings(sliders): Legge il file e aggiorna config.py, con validazione dei valori (es. alpha_smooth è limitato tra 0.1 e 1.0).

Elementi Grafici

Slider Tkinter per regolare parametri come sensibilità (alpha_smooth), velocità cursore (cursor_speed_multiplier), ecc.

Pulsanti "Reset", "Salva", e "Info" con popup per le gesture.

Dettagli Implementativi

Mapping dei Valori: Gli slider utilizzano scale lineari (es. alpha_smooth è mappato da 1-10 a 0.05-0.5).

Gestione File: Se la cartella Profiles o il file settings.txt non esistono, vengono creati con valori di default.

hand_tracker

hand_tracker.py

Elabora i frame della webcam, rileva gesture e controlla il mouse. Rileva fino a 2 mani con un modello che ha fino 3 gradi di complessità.

Flusso Principale

find_hands rileva le mani e le mappa con *find_position*, poi vede quali dita sono alzate con *fingers_up*. Ci sono altre funzioni per vedere l'orientamento, calcolare distanza tra due punti, e gestire lo zoom.

Elaborazione Frame

Rilevamento Mani: `hands.process(rgb)` restituisce landmarks per ogni mano.

Ottimizzazioni

Cooldown: Timer per evitare azioni ripetute (es. `click_cooldown`, `zoom_cooldown_time`).

Smoothing: Media mobile su `zoom_distances` per stabilizzare lo zoom.

mouse_controller

mouse_controller.py

Usa il filtro di Kalman per smoothing ed inoltre sono presenti le seguenti gestione:

- Distanza indice-pollice
- Timers per click/drag
- Riconoscimento click destro via mignolo

gesture_recognizer

gesture_recognizer.py

È presente la gestione dello zoom a due mani con soglia dinamica, lo slide direzionale con pugno chiuso e il sistema di cooldown integrato.

gui_theme

gui_theme.py

Centralizza le scelte grafiche per uniformare il più possibile le due GUI (webcam selector e settings).

Parametri

GUI_FONT, GUI_FONT_BOLD: Font per testo e titoli.
Colori (es. *GUI_BG_COLOR*="#4a3f35" per lo sfondo).
Dimensioni finestre (*GUI_WINDOW_SIZE*="300x300").

config

config.py

Contiene variabili globali modificate dinamicamente da settings_gui.py:

Parametri controllo del mouse

alpha_smooth: Fattore di smoothing per il movimento del cursore (0.1-1.0).
overscan_x, overscan_y: Sensibilità del movimento orizzontale/verticale.
click_cooldown: Tempo minimo tra click consecutivi.
cursor_speed_multiplier: Moltiplicatore di velocità del cursore

Parametri per il riconoscimento dei gesti

click_cooldown: Tempo di attesa tra clic consecutivi
click_distance_threshold: Soglia di distanza per il riconoscimento del clic

Parametri per lo zoom

zoom_threshold: Soglia per il riconoscimento dello zoom
zoom_max_distance: Distanza massima per lo zoom

Parametri per lo slide

slide_cooldown_time :Tempo di cooldown per lo slide

Parametri per le prestazioni

use_hardware_acceleration :Accelerazione hardware

camera_width : Larghezza della risoluzione della camera

camera_height : Altezza della risoluzione della camera

model_complexity: Complessità del modello (0 = leggero, 1 = pesante)

min_detection_confidence : Confidenza minima per il rilevamento delle mani

min_tracking_confidence :Confidenza minima per il tracciamento delle mani

Flusso di esecuzione

Inizializzazione

- Selezione webcam e avvio GUI impostazioni
- Configurazione MediaPipe Hands
- Avvio del loop principale

Pipeline di elaborazione

- Rilevamento landmark (21 punti sulla mano)
- Classificazione gesti (dita alzate/abbassate)
- Mappatura coordinate a schermo
- Esecuzione azioni (move/click/scroll/drag)

Soluzioni adottate

Multi-threading:

- GUI non blocca l'acquisizione video
- Ricerca webcam in background

Performance:

- Livelli complessità modello (0=light, 1=full)
- Accelerazione hardware opzionale

Usabilità:

- Sistema di help integrato
- Reset a valori default

Estensibilità:

- Architettura modulare
- File di configurazione esterno
- API chiare tra componenti